

Price Per Token Guide to Using Fewer Tokens

Most developers use 2-5x more tokens than they need to. Here's how to fix that.

This guide covers eight practical ways to cut your token usage. Each one is independent — you can start with whichever applies to you — but they compound when used together.

Tip 1: Trim Your System Prompt

Your system prompt gets sent with every single API call. There's no getting around that if you're using an LLM as an assistant because each call is stateless- all the context needs to be included in every input. It is worth spending time on a frequently used system prompt for this reason

Most system prompts are far longer than they need to be. Common sources of bloat:

- **"You are a helpful assistant that..."** — The model already knows this. Cut it.
- **Redundant formatting instructions** — Telling the model three different ways to use bullet points doesn't help more than telling it once.
- **Politeness and motivation** — "Please always try your best to provide accurate..." is filler. LLMs don't try harder because you said please.
- **Reference material** — If you're pasting documentation into the system prompt, move it to retrieval (RAG) or tool calls instead. It doesn't belong in every request.

The rule of thumb: Remove a line from your system prompt. If the model's behavior doesn't change, leave it out.

Example:

Before (roughly 800 tokens):

```
You are a helpful, knowledgeable AI assistant specializing in customer support for our e-commerce platform. You should always be polite, professional, and thorough in your responses. When a customer asks about an order, you should look up their order details and provide a comprehensive summary including the order date, items ordered, shipping status, estimated delivery date, and any relevant tracking information. You should always ask if there's anything else you can help with at the end of your response. If you don't know the answer,
```

say so honestly rather than making something up. Format your responses clearly with bullet points where appropriate...

After (roughly 200 tokens):

Customer support agent for [Company]. Look up order details when asked. Be concise. If unsure, say so. Use the order_lookup tool for order queries.

Same behavior. 600 fewer tokens per call. At 200 calls/day, that's 120,000 tokens saved daily — purely from deleting words the model didn't need. Details about looking up order dates, items ordered, etc can be included in tool definitions and picked up into context only when necessary.

Tip 2: Control Your Output Length

If you don't tell the model how long to respond, it will use as many tokens as it wants. This matters because output tokens are the expensive ones — typically 3-5x the cost of input tokens.

A few ways to control this:

Set `max_tokens` explicitly. Most people leave this at the default, which is often the model's maximum context window. If you only need a one-paragraph answer, don't leave room for an essay.

Be specific in your instructions: - "Respond in 2-3 sentences" works better than "be brief" - "Keep your answer under 100 words" gives a hard target - "Return only the category name, nothing else" eliminates all filler

Ask for bullets instead of prose. The same information in bullet point format uses fewer tokens than full paragraphs. If you're going to extract the key points anyway, have the model do it upfront.

For classification and routing tasks, constrain the output to a single word or a short JSON value. There's no reason a sentiment classifier should return "The sentiment of this text is positive because the author expresses satisfaction with..." when it could return `"positive"`.

The math: If a typical response is 2,000 tokens and you can bring it down to 500, you're saving 75% on the most expensive part of every call. Check what output tokens cost for your model at pricper.token.com — the difference between controlled and uncontrolled output adds up fast.

Tip 3: Use Structured Outputs

When you need data back from a model — not conversation, not creative writing, just data — structured outputs are the most token-efficient way to get it.

JSON mode and structured output schemas (available in OpenAI and Anthropic APIs) constrain the model's response to match a predefined format. That means:

- No "Sure! Here's what I found:" preamble
- No trailing "Let me know if you need anything else!"
- No inconsistent formatting between calls
- Just the fields you asked for, in the shape you defined

Structured responses are typically 30-60% shorter than free-form text containing the same information. And they're easier to parse — no regex, no "sometimes it includes a header and sometimes it doesn't."

When to use structured outputs: - Data extraction (pull name, date, amount from text) - Classification (return a category label, not an explanation of why) - Multi-step pipelines (pass clean data between steps) - Any time you're going to parse the response programmatically

Pydantic AI is worth looking at if you're working in Python. It lets you define your output schema as a Pydantic model, and the LLM is forced to return data matching that schema. No parsing code, no surprises, no wasted tokens on filler text. It works across multiple providers including OpenAI and Anthropic.

I personally strictly Pydantic AI for almost all use cases (I am not affiliated). It allows you to define each part of an output which in turn gives you predictable output formats at a controlled length.

Tip 4: Cache What Doesn't Change

Prompt caching gives you a 90% discount on tokens the API has already processed. If your prompts have a stable prefix — and most do — this is essentially free savings.

How it works: The first time you send a prompt, you pay full price. On subsequent requests within the cache window (5 minutes for Anthropic, similar for OpenAI), any tokens that match the cached prefix cost 90% less.

The key is prompt structure. The cached portion must be identical character-for-character from the start of the prompt. That means:

- Put stable content first: system prompt, few-shot examples, reference docs
- Put variable content last: the user's actual message, dynamic context
- Don't inject timestamps, request IDs, or session metadata into your system prompt — anything dynamic in the prefix invalidates the cache. If you must put dynamic information into the system prompt, put it at the end of it.

What this looks like in practice:

Say your system prompt is 2,000 tokens and you make 100 calls per day.

Without caching: 200,000 input tokens/day at full price.

With caching: 2,000 tokens at full price (first call) + 198,000 tokens at 90% off. That's a massive reduction for zero code changes — just making sure your prompt structure is cache-friendly.

To maximize cache hits: - Batch related requests within 5-minute windows so they all hit the cache - Don't update system prompt files mid-session — changes invalidate the cache - Keep reference docs in the prompt prefix (stable) and user queries at the end (variable)

Tip 5: Summarize Instead of Stuffing

One of the most common token sinks is cramming too much raw content into context. Full conversation histories, entire retrieved documents, verbose outputs from previous pipeline steps — all of it costs tokens, and most of it is noise.

Conversation history:

Don't pass your entire chat transcript with every message. Keep the last 3-5 messages verbatim (the model needs recent context) and summarize everything before that. A 200-token summary of what was discussed earlier replaces 5,000+ tokens of raw message history.

This is especially important for chatbots and agents that run long sessions. Without summarization, context grows linearly with every exchange, and you're paying full input token price on the entire history every single turn.

RAG and document retrieval:

When retrieving documents to answer a question, less is more:

- Retrieve 3 highly relevant chunks, not 10 "maybe relevant" ones
- Use smaller chunk sizes — a 200-token chunk wastes less space than a 1,000-token chunk when only one sentence in it is actually relevant
- Pre-filter results before sending them to the LLM. Your vector search's relevance scores can eliminate weak matches before they eat context tokens

Multi-step pipelines:

If you're chaining multiple LLM calls, don't pass the full raw output of step 1 into step 2. Extract only what the next step needs and pass a structured summary. A 50-token JSON object is cheaper than a 500-token paragraph that says the same thing.

Tip 6: Pick the Right Model for the Job

Many people jump straight to the popular closed-source models like Claude and GPT. However, open-source models are catching up in terms of performance and have much lower token costs. I personally have been leaning heavily on Kimi K2.5 and recommend you try it as well, it a fraction of the cost as using Claude Sonnet with similar results from my experience.

Tip 7: Limit Tool Calls (Especially with RAG)

If you give an LLM access to search or retrieval tools without limits, it will use them far more than necessary. This is one of the biggest hidden token costs in agentic systems.

The loop looks like this: The model searches for something. Gets results back. Decides it needs more information. Searches again. Gets more results. Decides it should check one more thing. Searches again. By the time it finally answers, it's burned through 5+ retrieval calls when 1-2 would have been enough.

Each round-trip costs you four times over: 1. The model's reasoning tokens (deciding what to search) 2. The function call tokens (the search query itself) 3. The returned results tokens (the content that comes back) 4. The model's interpretation tokens (reading and processing the results)

Five unnecessary retrieval calls can easily burn 10,000+ tokens. And it compounds — each set of results gets added to context, so every subsequent call is more expensive than the last. A search loop that starts with 2,000 context tokens can end at 15,000.

How to prevent this:

Set a hard cap. Tell the model in the system prompt: "You have a maximum of 3 search calls. Use them wisely and answer with what you have." Most models respect this when the instruction is clear. You can send warnings to the model as it uses tools that it is running out of calls and to wrap up its searching.

Pre-filter your retrieval. If your vector search returns 20 chunks, trim to the top 3 before passing them to the LLM. Don't make the model pay tokens to read content that wasn't relevant enough to rank highly.

Set token limits on tool results. If a retrieved document is 5,000 tokens long, truncate it. The model rarely needs the entire document — usually a few hundred tokens contain the relevant answer.

I've tested RAG pipelines with unlimited tool calls and was shocked when one of these times I had spent \$15 on a single run. I set search call limits to 12 and since then can run the pipeline for less than a dollar with similar results as unlimited search.

Tip 8: Batch Your Requests

Every API call carries overhead. Your system prompt, any few-shot examples, and the conversation setup all get sent every time. If you're making the same type of call repeatedly, batching multiple items into a single request means you pay that overhead once instead of ten times.

Instead of this (10 API calls):

```
Call 1: "Classify this email: [email 1]"
Call 2: "Classify this email: [email 2]"
...
Call 10: "Classify this email: [email 10]"
```

Do this (1 API call):

```
"Classify each of the following emails:
1. [email 1]
2. [email 2]
..."
```

```
10. [email 10]
```

```
Return a JSON array with the classification for each."
```

Your system prompt tokens get sent once instead of ten times. That alone saves thousands of tokens at scale.

Batch APIs take this further. OpenAI's Batch API offers a 50% discount for requests that don't need real-time responses. If you're processing data overnight, running evaluations, or doing any kind of bulk work, batch endpoints are significantly cheaper.

When batching makes sense: - Processing lists of items (emails, support tickets, documents) - Non-real-time workloads (nightly data processing, weekly reports) - Evaluation and testing (running your prompt against many test cases) - Any repetitive task where latency isn't critical

When it doesn't: Real-time user-facing responses where each message is unique and needs immediate replies.

Compare model pricing and research LLMs at pricepertoken.com.